learn in-context (Von Oswald et al., 2023; Dai et al., 2022). The work of Li et al. (2023) presents a theoretical framework for studying ICL, providing some risk bounds on ICL of supervised learning algorithms. Guo et al. (2023) construct synthetic in-context learning problems with a compositional structure, studying the representation capacity of transformers to learn these problems in-context. In (Hendel et al., 2023), the authors demonstrate that transformers learn to represent task vectors, providing a mechanistic analysis of ICL in LLMs. Kirsch et al. (2022) view ICL as a broad meta-learning paradigm, and observe that transformers meta-trained on real image classification tasks undergo similar phase transitions as the ones we observe in this work.

Induction Heads. Elhage et al. (2021) studies the formation of induction heads, sub-components of transformers that match previous occurrences of the current token, retrieving the token that succeeds the most recent occurrence. Olsson et al. (2022) studies in-context learning by analyzing the formation of induction heads in language models, showing empirical evidence that both large and small transformers display a phase transition in the ability to learn in-context. Reddy (2023) also studies the formation of induction heads and their role in ICL, showing empirically that a three layer network exhibits a sudden formation of induction heads towards solving some ICL problem of interest. Bietti et al. (2023) study the effect of specific trigger tokens on the formation of induction heads.

Phase Transitions. It has been observed in different contexts that neural networks and language models display a sudden drop in loss during their training process. This phase transition is often related to emergence of new capabilities in the network. The work of (Power et al., 2022) observed the "grokking" phenomena, where the test loss of neural networks sharply drops, long after the network overfits the training data. (Chen et al., 2023) shows another example of a phase transition in language model training, where the formation of specific attention mechanisms happen suddenly in training, causing the loss to quickly drop. Barak et al. (2022) observe that neural networks trained on complex learning problems display a phase transition when converging to the correct solution. Several works (Kumar et al., 2023; Lyu et al., 2023) attribute these phase transitions to rapid changes in the inductive bias of networks, while Merrill et al. (2023) argue that the models are sparser after the phase change. Schaeffer et al. (2023) warn that phenomena in deep learning that seem to be discontinuous can actually be understood to evolve continuously once seen through the right lens.

Simplicity Bias. Various works observed that neural networks have a "simplicity bias", which causes them to "prioritize" learning simple patterns first (Arpit et al., 2017; Valle-Perez et al., 2018). The work of (Kalimeris et al., 2019) shows that SGD learns functions of increased complexity, first fitting a linear concept to the data before moving to more complex functions. (Shah et al., 2020) shows that the simplicity bias of neural networks can sometimes be harmful, causing them to ignore important features of the data. Chen et al. (2023) demonstrate the effect of simplicity bias on language tasks that require understanding of syntactic structure. Abbe et al. (2023) provide a theoretical framework for understanding how the simplicity of the target function can govern the convergence time of SGD, describing how simple partial solutions can speed up learning; in contrast, in our setting, the unigram solution appears likely to be a distractor which delays learning of the correct solution.

Concurrent works In parallel to this work, there have been a number of papers devoted to the study of similar questions regarding in-context learning and Markov Chains: Akyürek et al. (2024) empirically compare the ability of different architectures to perform in-context learning of regular languages. Hoogland et al. (2024) observe similar stage-wise learning behaviors on transformers trained on language or synthetic linear regression tasks. Makkuva et al. (2024) study the loss landscape of transformers trained on sequences sampled from a single Markov Chain.

2 Setup

In this section, we describe our learning problem, several analytical properties of it, and present the neural networks that we will use for learning.

2.1 ICL-MC Task

Our learning task consists of Markov Chains with random transition matrices. The goal is to in-context estimate the transition probabilities from sampled sequences, in order to predict the next state. Formally, a sample is a Markov Chain with state space $S = \{1, \ldots, k\}$ and a transition matrix $\mathcal{P}$ randomly sampled from some prior distribution, with x_1 drawn from some other prior distribution (potentially dependent on $\mathcal{P}$), and the rest of $\boldsymbol{x} = (x_1, \ldots, x_t)$ drawn from the Markov Chain. We primarily focus on the case where each row of the matrix is sampled from the Dirichlet distribution with concentration parameter $\boldsymbol{\alpha}$, i.e. $\mathcal{P}_{i,:} \sim \mathrm{Dir}(\boldsymbol{\alpha})$. We want to learn a predictor that, given context $x_1, \ldots, x_t$, predicts the next token, x_{t+1}. Note that this is an inherently non-deterministic task, even provided full information about the transition matrix, and as such it can better capture certain properties of language than previous in-context learning modeling approaches (Garg et al., 2022).

We focus on the case of $\boldsymbol{\alpha} = (1, \ldots, 1)^\top$ that corresponds to uniformly random transition probabilities between states. We draw the initial state x_1 from the stationary distribution $\boldsymbol{\pi}$ of the chain (which exists almost surely). We primarily consider the case where the number of states k is 2 or 3.

In subsection 3.3, we consider the generalization of this setting to n-grams for $n > 2$. Instead of $\Pr(x_t)$ being determined by x_{t-1}, we let $\Pr(x_t)$ be determined by $x_{t-n+1}, \ldots, x_{t-1}$, according to a conditional distribution $\mathcal{P}$ drawn from some prior. In particular, for each tuple of $n-1$ tokens, we sample the vector of conditional probabilities for the next state from a uniform Dirichlet distribution.

2.2 Potential Strategies to Solve ICL-MC

We adopt the Bayesian interpretation of in-context learning (Xie et al., 2022), in which a prior distribution is given by the training data, and, at test time, the model updates this prior given the in-context sequence. In this framework, we focus on two strategies for Bayesian inference: a *unigram* strategy which assumes tokens in each sequence are i.i.d. samples, and the *bigram* strategy which correctly takes into account dependencies among adjacent tokens.

1st strategy: Unigrams Since we let the Markov chain reach its stationary distribution (which exists a.s.), the optimal strategy across unigrams is just to count frequency of states and form a posterior belief about the stationary distribution. Unfortunately, the stationary distribution of this random Markov chain does not admit a simple analytical characterization when there is a finite number of states, but it can be estimated approximately. At the limit of $k \to \infty$, the stationary distribution converges to the uniform distribution (Bordenave et al., 2008).

2nd strategy: Bigrams For any pair of states i and j, let $\mathcal{P}_{ij}$ be probability of transitioning from i to j. On each sample $\boldsymbol{x}$, we can focus on the transitions from the i-th state, which follow a categorical distribution with probabilities equal to $(\mathcal{P}_{i1}, \ldots, \mathcal{P}_{ik})$. If we observe the in-context empirical counts $\{c_{ij}\}_{j=1}^k$ of the transitions, then $\mathcal{P}_{ij}$ is given by:

$$(\mathcal{P}_{i1}, \ldots, \mathcal{P}_{ik}) \,|\boldsymbol{x} \sim \mathrm{Dir}(k, c_{i1} + \alpha_1, \ldots, c_{ik} + \alpha_k), \tag{1}$$

where, recall, $\alpha_1, \ldots, \alpha_k$ are the Dirichlet concentration parameters of the prior. Hence, each $\mathcal{P}_{ij}$ has a (marginal) distribution that is actually a Beta distribution:

$$\mathcal{P}_{ij}|\boldsymbol{x} \sim \mathrm{Beta}\left(c_{ij} + \alpha_j, \sum_j \alpha_j + N_i - \alpha_j - c_{ij}\right), \tag{2}$$

where N_i is the total number of observed transitions from state i. As such, our best (point) estimate for each state j is given by:

$$\mathbb{E}\left[\mathcal{P}_{ij}|\boldsymbol{x}\right] = \frac{c_{ij} + \alpha_j}{N + \sum_i \alpha_i}. \tag{3}$$

For the uniform Dirichlet, $\boldsymbol{\alpha} = (1, \ldots, 1)^\top$, it is $\mathbb{E}\left[\mathcal{P}_{ij}|\boldsymbol{x}\right] = \frac{c_{ij}+1}{N_i+k}$.

Remark 2.1. The bigram strategy implicitly assumes that the first token x_1 is sampled uniformly, as opposed to being sampled from the stationary distribution (which is used in our experiments and theoretical results). As the context length grows, the bigram statistics approach the Bayes optimal solution either way and this difference becomes negligible.

2.3 Architectures: Transformers and Simplifications

We are mainly interested in investigating how transformers (Vaswani et al., 2017) can succeed in in-context learning this task. We focus on attention-only transformers with 2 layers with causal masking which is a popular architecture for language modeling. Given an input sequence $\boldsymbol{x}$, the output of an n-layer attention-only transformer is:

$$TF(\boldsymbol{x}) = P \circ (Attn_1 + I) \cdots \circ (Attn_n + I) \tag{4}$$

Where $P \in \mathbb{R}^{k \times d}$ is a linear projection, $\mathbf{e}_{\boldsymbol{x}} \in \mathbb{R}^{t \times d}$ is an embedding of $\boldsymbol{x}$, and $Attn(\boldsymbol{x})$ is masked self attention with relative position embeddings (Shaw et al., 2018), which is parameterized by $W_Q, W_K, W_V \in \mathbb{R}^{k \times d}, v \in \mathbb{R}^{t \times d}$:

$$\begin{aligned} Attn(\mathbf{e}) &= \text{softmax}(\text{mask}(A))\mathbf{e}W_V \\ A_{i,j} &= \frac{(\mathbf{e}_i W_Q + v_{i-j+1})(\mathbf{e}_j W_K)^\top}{\sqrt{d}}. \end{aligned} \tag{5}$$

During training, we minimize this loss:

$$L(\theta) = \mathop{\mathbb{E}}_{\substack{\boldsymbol{x} \sim \mathcal{P} \\ \mathcal{P} \sim \text{Dir}(\boldsymbol{\alpha})^{\otimes k}}} \left[\frac{1}{t} \sum_{p=1}^{t} l\left(TF(\boldsymbol{x}; \theta)_p, x_{p+1}\right) \right], \tag{6}$$

where θ denotes the parameters of the model and l is a loss function such as the cross entropy or margin loss. For our experiments, we run on the standard cross-entropy loss. For our theoretical results, we analyze training under the margin loss that is a generalization of the hinge loss to the case of more than 2 classes: for hyperparameter $\Delta > 0$,

$$l_M(f(e)_{p,:}, x_{p+1}) = \frac{1}{k} \sum_{\substack{i=1, \\ i \neq x_{p+1}}}^{k} \max\left\{0, \Delta + f(e)_{p,i} - f(e)_{p,x_{p+1}}\right\}. \tag{7}$$

We now show how a two-layer transformer can represent the optimal bigrams solution.

Proposition 2.2 (Transformer Construction)**.** *A single-head two layer attention-only transformer can find the bigram statistics in the in-context learning markov chain task.*

Proof. Set the internal dimension $d = 3k$, and choose $\mathbf{e}_{\boldsymbol{x}}$ to be one-hot embeddings, that is, $\mathbf{e}_{\boldsymbol{x}_i} = \delta_{\boldsymbol{x}_i}$, where δ is the Kronecker delta. We will call the parameters of attention layer i, $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}, v^{(i)}$. Let

$$v^{(1)} = \delta_2 \mathbf{1}^\top \qquad W_Q^{(1)} = \mathbf{0} \qquad W_K^{(1)} = \begin{pmatrix} cI^{k \times k} & \mathbf{0} & \mathbf{0} \end{pmatrix} \qquad W_V^{(1)} = \begin{pmatrix} \mathbf{0} & I^{k \times k} & \mathbf{0}. \end{pmatrix}$$

The attention of this first layer will focus on the previous token, $A_{i,i-1} = c$ and for all $j \neq i-1$, $A_{i,j} = \mathbf{0}$. Then, as c approaches infinity, the first attention layer at index i approaches $\begin{pmatrix} \mathbf{0} & \mathbf{e}_{\boldsymbol{x}_{i-1}} & \mathbf{0} \end{pmatrix}$.

$$v^{(2)} = \mathbf{0} \qquad W_Q^{(2)} = \begin{pmatrix} cI^{k \times k} & \mathbf{0} & \mathbf{0} \end{pmatrix} \qquad W_K^{(2)} = \begin{pmatrix} \mathbf{0} & I^{k \times k} & \mathbf{0} \end{pmatrix} \qquad W_V^{(2)} = \begin{pmatrix} \mathbf{0} & \mathbf{0} & I^{k \times k}. \end{pmatrix}$$